

折纸小鸟对对碰 里程碑 1 抄写指南：纯 C++ 三消内核

本文档只指导你手动抄代码，不要求我直接改源码。

目标：先不接 UI、不接活动入口、不接 GAS，只把 7x7 三消棋盘规则跑通，并能用日志做最小测试。

0. 本阶段要做什么

本阶段产物：

- 一个三消玩法模块目录： `Source/HoyoGas/OrigamiBird`
- 一个关卡配置表结构： `FOrigamiBirdMatchLevelRow`
- 一个玩法核心对象： `UOrigamiBirdMatchGameObject`
- 一个最小 Debug API： `DumpBoardToLog`

本阶段不做：

- UMG / MVVM
- 活动入口
- 存档
- 音频
- GAS
- 联机

但会预留接口：

- `StartParams`：未来可由活动入口、快捷键入口、关卡系统传入。
- `Command`：未来联机可以同步玩家输入命令。
- `Result`：未来接 Progression / SaveGame。
- `DynamicMulticastDelegate`：未来蓝图监听后播放音频、特效、动画。

1. 修改 Build.cs

打开：

```
Source/HoyoGas/HoyoGas.Build.cs
```

找到 `PublicIncludePaths.AddRange`，加入：

```
Path.Combine(ModuleDirectory, "OrigamiBird/Public"),
```

建议放在 `CharacterGrowth/Public` 后面。

找到 `PrivateIncludePaths.AddRange`，加入：

```
Path.Combine(ModuleDirectory, "OrigamiBird/Private"),
```

建议放在 `CharacterGrowth/Private` 后面。

2. 新建目录

手动创建这些目录：

```
Source/HoyoGas/OrigamiBird/Public/Core
Source/HoyoGas/OrigamiBird/Public/Data
Source/HoyoGas/OrigamiBird/Private/Core
```

3. 新建类型文件

新建文件：

```
Source/HoyoGas/OrigamiBird/Public/Core/OrigamiBirdMatchTypes.h
```

内容：

```
#pragma once

#include "CoreMinimal.h"
#include "OrigamiBirdMatchTypes.generated.h"

UENUM(BlueprintType)
enum class EOrigamiBirdTileType : uint8
{
    None,
    RedFruit,
    BlueFruit,
    YellowFruit,
    GreenFruit,
    PurpleFruit,
    RowBomb,
    ColumnBomb
};

UENUM(BlueprintType)
enum class EOrigamiBirdMatchPhase : uint8
{
    None,
    WaitingInput,
    Resolving,
    GameEnded
};

UENUM(BlueprintType)
enum class EOrigamiBirdMatchCommandType : uint8
{
    None,
```

```

    SelectTile,
    SwapTiles,
    Restart
};

USTRUCT(BlueprintType)
struct HOYOGAS_API ForigamiBirdTile
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 TileId = INDEX_NONE;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    FIntPoint BoardPosition = FIntPoint::ZeroValue;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    EOrigamiBirdTileType TileType = EOrigamiBirdTileType::None;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    bool bIsSelected = false;
};

USTRUCT(BlueprintType)
struct HOYOGAS_API ForigamiBirdMatchStartParams
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    FName LevelId = NAME_None;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 BoardWidth = 7;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 BoardHeight = 7;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 MoveLimit = 25;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 TargetScore = 3000;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 RandomSeed = 12345;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    TArray<EOrigamiBirdTileType> AvailableTileTypes;
};

USTRUCT(BlueprintType)
struct HOYOGAS_API ForigamiBirdMatchCommand
{

```

```

GENERATED_BODY()

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
EOrigamiBirdMatchCommandType CommandType = EOrigamiBirdMatchCommandType::None;

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
FIntPoint From = FIntPoint(INDEX_NONE, INDEX_NONE);

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
FIntPoint To = FIntPoint(INDEX_NONE, INDEX_NONE);
};

USTRUCT(BlueprintType)
struct HOYOGAS_API FOrigamiBirdBoardSnapshot
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 BoardWidth = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 BoardHeight = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    TArray<FOrigamiBirdTile> Tiles;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 Score = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 MovesRemaining = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 ComboCount = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    EOrigamiBirdMatchPhase Phase = EOrigamiBirdMatchPhase::None;
};

USTRUCT(BlueprintType)
struct HOYOGAS_API FOrigamiBirdMatchResult
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    FName LevelId = NAME_None;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 FinalScore = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
    int32 UsedMoves = 0;
};

```

```

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
int32 MaxCombo = 0;

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
int32 RemovedTileCount = 0;

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "OrigamiBird")
bool bSuccess = false;
};

```

4. 新建关卡数据表结构

新建文件:

Source/HoyoGas/OrigamiBird/Public/Data/OrigamiBirdMatchLevelRow.h

内容:

```

#pragma once

#include "CoreMinimal.h"
#include "Engine/DataTable.h"
#include "Core/OrigamiBirdMatchTypes.h"
#include "OrigamiBirdMatchLevelRow.generated.h"

USTRUCT(BlueprintType)
struct HOYOGAS_API FOrigamiBirdMatchLevelRow : public FTableRowBase
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "OrigamiBird")
    FName LevelId = NAME_None;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "OrigamiBird", meta = (ClampMin = "3", ClampMax = "12"))
    int32 Boardwidth = 7;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "OrigamiBird", meta = (ClampMin = "3", ClampMax = "12"))
    int32 BoardHeight = 7;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "OrigamiBird", meta = (ClampMin = "1"))
    int32 MoveLimit = 25;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "OrigamiBird", meta = (ClampMin = "1"))
    int32 TargetScore = 3000;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "OrigamiBird")
    int32 RandomSeed = 12345;
}

```

```
UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "OrigamiBird")
TArray<EOrigamiBirdTileType> AvailableTileTypes;
};
```

说明：

- 现在还不读 DataTable。
- 先把结构定义出来，后面里程碑 4 再接策划表。
- 第一版 `StartParams` 可以手动填 7x7。

5. 新建玩法核心头文件

新建文件：

```
Source/HoyoGas/OrigamiBird/Public/Core/OrigamiBirdMatchGameObject.h
```

内容：

```
#pragma once

#include "CoreMinimal.h"
#include "UObject/Object.h"
#include "OrigamiBirdMatchTypes.h"
#include "OrigamiBirdMatchGameObject.generated.h"

DECLARE_DYNAMIC_MULTICAST_DELEGATE(ForigamiBirdMatchSimpleEvent);
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(ForigamiBirdBoardChangedEvent, const
ForigamiBirdBoardSnapshot&, Snapshot);
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(ForigamiBirdTileSelectedEvent, FIntPoint,
BoardPosition);
DECLARE_DYNAMIC_MULTICAST_DELEGATE_TwoParams(ForigamiBirdSwapResolvedEvent, FIntPoint, From,
FIntPoint, To);
DECLARE_DYNAMIC_MULTICAST_DELEGATE_TwoParams(ForigamiBirdTilesMatchedEvent, const
TArray<FIntPoint>&, MatchedPositions, int32, ComboIndex);
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(ForigamiBirdScoreChangedEvent, int32, NewScore);
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(ForigamiBirdMovesChangedEvent, int32,
NewMovesRemaining);
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(ForigamiBirdGameEndedEvent, const
ForigamiBirdMatchResult&, Result);

UCLASS(BlueprintType)
class HOYOGAS_API UOrigamiBirdMatchGameObject : public UObject
{
    GENERATED_BODY()

public:
    UPROPERTY(BlueprintAssignable, Category = "OrigamiBird")
    ForigamiBirdBoardChangedEvent OnBoardChanged;

    UPROPERTY(BlueprintAssignable, Category = "OrigamiBird")
    ForigamiBirdTileSelectedEvent OnTileSelected;
```

```

UPROPERTY(BlueprintAssignable, Category = "OrigamiBird")
ForigamiBirdSwapResolvedEvent OnSwapResolved;

UPROPERTY(BlueprintAssignable, Category = "OrigamiBird")
ForigamiBirdTilesMatchedEvent OnTilesMatched;

UPROPERTY(BlueprintAssignable, Category = "OrigamiBird")
ForigamiBirdScoreChangedEvent OnScoreChanged;

UPROPERTY(BlueprintAssignable, Category = "OrigamiBird")
ForigamiBirdMovesChangedEvent OnMovesChanged;

UPROPERTY(BlueprintAssignable, Category = "OrigamiBird")
ForigamiBirdGameEndedEvent OnGameEnded;

UFUNCTION(BlueprintCallable, Category = "OrigamiBird")
void Initialize(const ForigamiBirdMatchStartParams& InStartParams);

UFUNCTION(BlueprintCallable, Category = "OrigamiBird")
bool SubmitCommand(const ForigamiBirdMatchCommand& Command);

UFUNCTION(BlueprintCallable, Category = "OrigamiBird")
bool SelectTile(FIntPoint BoardPosition);

UFUNCTION(BlueprintCallable, Category = "OrigamiBird")
bool TrySwapTiles(FIntPoint From, FIntPoint To);

UFUNCTION(BlueprintCallable, Category = "OrigamiBird")
ForigamiBirdBoardSnapshot GetSnapshot() const;

UFUNCTION(BlueprintCallable, Category = "OrigamiBird")
void DumpBoardToLog() const;

private:
    ForigamiBirdMatchStartParams StartParams;
    FRandomStream RandomStream;
    TArray<ForigamiBirdTile> BoardTiles;
    EOrigamiBirdMatchPhase Phase = EOrigamiBirdMatchPhase::None;
    FIntPoint SelectedPosition = FIntPoint(INDEX_NONE, INDEX_NONE);
    int32 Score = 0;
    int32 MovesRemaining = 0;
    int32 UsedMoves = 0;
    int32 MaxCombo = 0;
    int32 RemovedTileCount = 0;
    int32 NextTileId = 1;

    bool IsInsideBoard(FIntPoint Position) const;
    bool AreAdjacent(FIntPoint A, FIntPoint B) const;
    int32 ToIndex(FIntPoint Position) const;
    ForigamiBirdTile* GetTile(FIntPoint Position);
    const ForigamiBirdTile* GetTile(FIntPoint Position) const;
    void SwapTileData(FIntPoint A, FIntPoint B);

```

```

EOrigamiBirdTileType GenerateRandomTileType();
void GenerateInitialBoard();
bool WouldCreateMatchAt(FIntPoint Position) const;

TArray<FIntPoint> FindAllMatches() const;
void RemoveTiles(const TArray<FIntPoint>& Positions);
void CollapseAndRefill();
bool ResolveBoardAfterValidSwap();

void ClearSelection();
void BroadcastBoardChanged();
void CheckGameEnd();
};

```

6. 新建玩法核心 cpp

新建文件:

```
Source/HoyoGas/OrigamiBird/Private/Core/OrigamiBirdMatchGameObject.cpp
```

内容:

```

#include "Core/OrigamiBirdMatchGameObject.h"

DEFINE_LOG_CATEGORY_STATIC(LogOrigamiBirdMatch, Log, All);

void UOrigamiBirdMatchGameObject::Initialize(const FOrigamiBirdMatchStartParams&
InStartParams)
{
    StartParams = InStartParams;
    if (StartParams.AvailableTileTypes.IsEmpty())
    {
        StartParams.AvailableTileTypes =
        {
            EOrigamiBirdTileType::RedFruit,
            EOrigamiBirdTileType::BlueFruit,
            EOrigamiBirdTileType::YellowFruit,
            EOrigamiBirdTileType::GreenFruit,
            EOrigamiBirdTileType::PurpleFruit
        };
    }

    StartParams.BoardWidth = FMath::Clamp(StartParams.BoardWidth, 3, 12);
    StartParams.BoardHeight = FMath::Clamp(StartParams.BoardHeight, 3, 12);
    RandomStream.Initialize(StartParams.RandomSeed);

    BoardTiles.Reset();
    Score = 0;
    MovesRemaining = FMath::Max(1, StartParams.MoveLimit);
    UsedMoves = 0;
    MaxCombo = 0;
}

```



```

    RemovedTileCount = 0;
    NextTileId = 1;
    SelectedPosition = FIntPoint(INDEX_NONE, INDEX_NONE);
    Phase = EOrigamiBirdMatchPhase::WaitingInput;

    GenerateInitialBoard();
    BroadcastBoardChanged();
}

bool UOrigamiBirdMatchGameObject::SubmitCommand(const FOrigamiBirdMatchCommand& Command)
{
    switch (Command.CommandType)
    {
    case EOrigamiBirdMatchCommandType::SelectTile:
        return SelectTile(Command.From);
    case EOrigamiBirdMatchCommandType::SwapTiles:
        return TrySwapTiles(Command.From, Command.To);
    case EOrigamiBirdMatchCommandType::Restart:
        Initialize(StartParams);
        return true;
    default:
        return false;
    }
}

bool UOrigamiBirdMatchGameObject::SelectTile(FIntPoint BoardPosition)
{
    if (Phase != EOrigamiBirdMatchPhase::WaitingInput || !IsInsideBoard(BoardPosition))
    {
        return false;
    }

    ClearSelection();
    if (FOrigamiBirdTile* Tile = GetTile(BoardPosition))
    {
        Tile->bIsSelected = true;
        SelectedPosition = BoardPosition;
        OnTileSelected.Broadcast(BoardPosition);
        BroadcastBoardChanged();
        return true;
    }

    return false;
}

bool UOrigamiBirdMatchGameObject::TrySwapTiles(FIntPoint From, FIntPoint To)
{
    if (Phase != EOrigamiBirdMatchPhase::WaitingInput || !IsInsideBoard(From) ||
    !IsInsideBoard(To) || !AreAdjacent(From, To))
    {
        return false;
    }
}

```

```

Phase = EOrigamiBirdMatchPhase::Resolving;
ClearSelection();

SwapTileData(From, To);
OnSwapResolved.Broadcast(From, To);

const TArray<FIntPoint> Matches = FindAllMatches();
if (Matches.IsEmpty())
{
    SwapTileData(From, To);
    Phase = EOrigamiBirdMatchPhase::WaitingInput;
    BroadcastBoardChanged();
    return false;
}

--MovesRemaining;
++UsedMoves;
OnMovesChanged.Broadcast(MovesRemaining);

ResolveBoardAfterValidSwap();
Phase = EOrigamiBirdMatchPhase::WaitingInput;
BroadcastBoardChanged();
CheckGameEnd();
return true;
}

FOrigamiBirdBoardSnapshot UOrigamiBirdMatchGameObject::GetSnapshot() const
{
    FOrigamiBirdBoardSnapshot Snapshot;
    Snapshot.BoardWidth = StartParams.BoardWidth;
    Snapshot.BoardHeight = StartParams.BoardHeight;
    Snapshot.Tiles = BoardTiles;
    Snapshot.Score = Score;
    Snapshot.MovesRemaining = MovesRemaining;
    Snapshot.ComboCount = MaxCombo;
    Snapshot.Phase = Phase;
    return Snapshot;
}

void UOrigamiBirdMatchGameObject::DumpBoardToLog() const
{
    UE_LOG(LogOrigamiBirdMatch, Log, TEXT("Board %dx%d Score=%d Moves=%d Phase=%d"),
StartParams.BoardWidth, StartParams.BoardHeight, Score, MovesRemaining, static_cast<int32>
(Phase));
    for (int32 Y = 0; Y < StartParams.BoardHeight; ++Y)
    {
        FString Line;
        for (int32 X = 0; X < StartParams.BoardWidth; ++X)
        {
            const FOrigamiBirdTile* Tile = GetTile(FIntPoint(X, Y));
            const int32 TypeValue = Tile ? static_cast<int32>(Tile->TileType) : 0;
            Line += FString::Printf(TEXT("%d "), TypeValue);
        }
    }
}

```

```

        UE_LOG(LogOrigamiBirdMatch, Log, TEXT("%s"), *Line);
    }
}

bool UOrigamiBirdMatchGameObject::IsInsideBoard(FIntPoint Position) const
{
    return Position.X >= 0
        && Position.Y >= 0
        && Position.X < StartParams.BoardWidth
        && Position.Y < StartParams.BoardHeight;
}

bool UOrigamiBirdMatchGameObject::AreAdjacent(FIntPoint A, FIntPoint B) const
{
    const int32 Distance = FMath::Abs(A.X - B.X) + FMath::Abs(A.Y - B.Y);
    return Distance == 1;
}

int32 UOrigamiBirdMatchGameObject::ToIndex(FIntPoint Position) const
{
    return Position.Y * StartParams.BoardWidth + Position.X;
}

FOrigamiBirdTile* UOrigamiBirdMatchGameObject::GetTile(FIntPoint Position)
{
    return IsInsideBoard(Position) ? &BoardTiles[ToIndex(Position)] : nullptr;
}

const FOrigamiBirdTile* UOrigamiBirdMatchGameObject::GetTile(FIntPoint Position) const
{
    return IsInsideBoard(Position) ? &BoardTiles[ToIndex(Position)] : nullptr;
}

void UOrigamiBirdMatchGameObject::SwapTileData(FIntPoint A, FIntPoint B)
{
    FOrigamiBirdTile* TileA = GetTile(A);
    FOrigamiBirdTile* TileB = GetTile(B);
    if (!TileA || !TileB)
    {
        return;
    }

    Swap(TileA->TileType, TileB->TileType);
    Swap(TileA->TileId, TileB->TileId);
}

EOrigamiBirdTileType UOrigamiBirdMatchGameObject::GenerateRandomTileType()
{
    if (StartParams.AvailableTileTypes.IsEmpty())
    {
        return EOrigamiBirdTileType::RedFruit;
    }
}

```

```

    const int32 Index = RandomStream.RandRange(0, StartParams.AvailableTileTypes.Num() - 1);
    return StartParams.AvailableTileTypes[Index];
}

void UOrigamiBirdMatchGameObject::GenerateInitialBoard()
{
    BoardTiles.SetNum(StartParams.BoardWidth * StartParams.BoardHeight);

    for (int32 Y = 0; Y < StartParams.BoardHeight; ++Y)
    {
        for (int32 X = 0; X < StartParams.BoardWidth; ++X)
        {
            const FIntPoint Position(X, Y);
            FOrigamiBirdTile& Tile = BoardTiles[ToIndex(Position)];
            Tile.TileId = NextTileId++;
            Tile.BoardPosition = Position;
            Tile.bIsSelected = false;

            int32 Guard = 0;
            do
            {
                Tile.TileType = GenerateRandomTileType();
                ++Guard;
            }
            while (WouldCreateMatchAt(Position) && Guard < 50);
        }
    }
}

bool UOrigamiBirdMatchGameObject::WouldCreateMatchAt(FIntPoint Position) const
{
    const FOrigamiBirdTile* Tile = GetTile(Position);
    if (!Tile || Tile->TileType == EOrigamiBirdTileType::None)
    {
        return false;
    }

    if (Position.X >= 2)
    {
        const FOrigamiBirdTile* Left1 = GetTile(FIntPoint(Position.X - 1, Position.Y));
        const FOrigamiBirdTile* Left2 = GetTile(FIntPoint(Position.X - 2, Position.Y));
        if (Left1 && Left2 && Left1->TileType == Tile->TileType && Left2->TileType == Tile->TileType)
        {
            return true;
        }
    }

    if (Position.Y >= 2)
    {
        const FOrigamiBirdTile* Up1 = GetTile(FIntPoint(Position.X, Position.Y - 1));
        const FOrigamiBirdTile* Up2 = GetTile(FIntPoint(Position.X, Position.Y - 2));
    }
}

```

```

        if (Up1 && Up2 && Up1->TileType == Tile->TileType && Up2->TileType == Tile-
>TileType)
        {
            return true;
        }
    }

    return false;
}

TArray<FIntPoint> UOrigamiBirdMatchGameObject::FindAllMatches() const
{
    TSet<FIntPoint> MatchedSet;

    for (int32 Y = 0; Y < StartParams.BoardHeight; ++Y)
    {
        int32 RunStartX = 0;
        for (int32 X = 1; X <= StartParams.BoardWidth; ++X)
        {
            const EOrigamiBirdTileType RunType = GetTile(FIntPoint(RunStartX, Y))->TileType;
            const bool bContinueRun = X < StartParams.BoardWidth && GetTile(FIntPoint(X,
Y))->TileType == RunType && RunType != EOrigamiBirdTileType::None;
            if (bContinueRun)
            {
                continue;
            }

            const int32 RunLength = X - RunStartX;
            if (RunType != EOrigamiBirdTileType::None && RunLength >= 3)
            {
                for (int32 MatchX = RunStartX; MatchX < X; ++MatchX)
                {
                    MatchedSet.Add(FIntPoint(MatchX, Y));
                }
            }
            RunStartX = X;
        }
    }

    for (int32 X = 0; X < StartParams.BoardWidth; ++X)
    {
        int32 RunStartY = 0;
        for (int32 Y = 1; Y <= StartParams.BoardHeight; ++Y)
        {
            const EOrigamiBirdTileType RunType = GetTile(FIntPoint(X, RunStartY))->TileType;
            const bool bContinueRun = Y < StartParams.BoardHeight && GetTile(FIntPoint(X,
Y))->TileType == RunType && RunType != EOrigamiBirdTileType::None;
            if (bContinueRun)
            {
                continue;
            }

            const int32 RunLength = Y - RunStartY;

```

```

        if (RunType != EOrigamiBirdTileType::None && RunLength >= 3)
        {
            for (int32 MatchY = RunStartY; MatchY < Y; ++MatchY)
            {
                MatchedSet.Add(FIntPoint(X, MatchY));
            }
        }
        RunStartY = Y;
    }
}

TArray<FIntPoint> MatchedPositions = MatchedSet.Array();
MatchedPositions.Sort([](const FIntPoint& A, const FIntPoint& B)
{
    return A.Y == B.Y ? A.X < B.X : A.Y < B.Y;
});
return MatchedPositions;
}

void UOrigamiBirdMatchGameObject::RemoveTiles(const TArray<FIntPoint>& Positions)
{
    for (const FIntPoint Position : Positions)
    {
        if (FOrigamiBirdTile* Tile = GetTile(Position))
        {
            Tile->TileType = EOrigamiBirdTileType::None;
            Tile->TileId = INDEX_NONE;
            Tile->bIsSelected = false;
        }
    }
}

void UOrigamiBirdMatchGameObject::CollapseAndRefill()
{
    for (int32 X = 0; X < StartParams.BoardWidth; ++X)
    {
        int32 WriteY = StartParams.BoardHeight - 1;
        for (int32 ReadY = StartParams.BoardHeight - 1; ReadY >= 0; --ReadY)
        {
            FOrigamiBirdTile* ReadTile = GetTile(FIntPoint(X, ReadY));
            if (ReadTile && ReadTile->TileType != EOrigamiBirdTileType::None)
            {
                if (WriteY != ReadY)
                {
                    FOrigamiBirdTile* WriteTile = GetTile(FIntPoint(X, WriteY));
                    WriteTile->TileType = ReadTile->TileType;
                    WriteTile->TileId = ReadTile->TileId;
                    WriteTile->bIsSelected = false;
                    ReadTile->TileType = EOrigamiBirdTileType::None;
                    ReadTile->TileId = INDEX_NONE;
                    ReadTile->bIsSelected = false;
                }
                --WriteY;
            }
        }
    }
}

```

```

    }
}

for (int32 Y = writeY; Y >= 0; --Y)
{
    ForigamiBirdTile* Tile = GetTile(FIntPoint(X, Y));
    Tile->TileType = GenerateRandomTileType();
    Tile->TileId = NextTileId++;
    Tile->bIsSelected = false;
}
}

for (int32 Y = 0; Y < StartParams.BoardHeight; ++Y)
{
    for (int32 X = 0; X < StartParams.BoardWidth; ++X)
    {
        if (ForigamiBirdTile* Tile = GetTile(FIntPoint(X, Y)))
        {
            Tile->BoardPosition = FIntPoint(X, Y);
        }
    }
}

}

bool UOrigamiBirdMatchGameObject::ResolveBoardAfterValidSwap()
{
    bool bResolvedAnyMatch = false;
    int32 ComboIndex = 0;

    while (true)
    {
        const TArray<FIntPoint> Matches = FindAllMatches();
        if (Matches.IsEmpty())
        {
            break;
        }

        bResolvedAnyMatch = true;
        ++ComboIndex;
        MaxCombo = FMath::Max(MaxCombo, ComboIndex);
        RemovedTileCount += Matches.Num();

        OnTilesMatched.Broadcast(Matches, ComboIndex);

        const float ComboMultiplier = ComboIndex == 1 ? 1.0f : (ComboIndex == 2 ? 1.2f :
(ComboIndex == 3 ? 1.5f : 2.0f));
        Score += FMath::RoundToInt(static_cast<float>(Matches.Num()) * 10.0f *
ComboMultiplier);
        OnScoreChanged.Broadcast(Score);

        RemoveTiles(Matches);
        CollapseAndRefill();
    }
}

```

```

        return bResolvedAnyMatch;
    }

    void UOrigamiBirdMatchGameObject::ClearSelection()
    {
        for (FOrigamiBirdTile& Tile : BoardTiles)
        {
            Tile.bIsSelected = false;
        }
        SelectedPosition = FIntPoint(INDEX_NONE, INDEX_NONE);
    }

    void UOrigamiBirdMatchGameObject::BroadcastBoardChanged()
    {
        OnBoardChanged.Broadcast(GetSnapshot());
    }

    void UOrigamiBirdMatchGameObject::CheckGameEnd()
    {
        const bool bSuccess = Score >= StartParams.TargetScore;
        const bool bFailed = MovesRemaining <= 0;
        if (!bSuccess && !bFailed)
        {
            return;
        }

        Phase = EOrigamiBirdMatchPhase::GameEnded;

        FOrigamiBirdMatchResult Result;
        Result.LevelId = StartParams.LevelId;
        Result.FinalScore = Score;
        Result.UsedMoves = UsedMoves;
        Result.MaxCombo = MaxCombo;
        Result.RemovedTileCount = RemovedTileCount;
        Result.bSuccess = bSuccess;

        OnGameEnded.Broadcast(Result);
        BroadcastBoardChanged();
    }

```

7. 最小测试方式 A：临时蓝图测试

先不做 UI。你可以临时在任意蓝图里：


```
Construct Object From Class
Class = OrigamiBirdMatchGameobject

Make OrigamiBirdMatchStartParams
Boardwidth = 7
BoardHeight = 7
MoveLimit = 25
TargetScore = 3000
RandomSeed = 12345

Initialize
DumpBoardToLog
```

然后运行 PIE，看 Output Log。

8. 最小测试方式 B：临时 C++ 调用

如果你想加 GM 指令，我们下一步再做一个更干净的 CheatManager 或 Exec 入口。

这一阶段先不急着想入口，因为核心类编译通过后，蓝图就已经能创建和 Dump。

9. 编译命令

保存文件后执行：

```
& 'D:\UE554\UE_5.5\Engine\Build\BatchFiles\Build.bat' HoyoGasEditor Win64 Development -
Project='D:\UE554\MyProjects\MihoyoGas\HoyoGas\HoyoGas.uproject' -waitmutex -NoHotReload
```

10. 本阶段验收标准

编译通过后，创建 UOrigamiBirdMatchGameobject，调用：

```
Initialize
DumpBoardToLog
```

能看到 7 行棋盘日志，每行 7 个数字。

然后随便用蓝图调用：

```
TrySwapTiles
```

如果相邻交换产生三消：

- 分数增加。
- 步数减少。
- 棋盘刷新。

如果不产生三消：

- 棋盘交换回去。
- 步数不减少。

11. 做完后提交

编译通过后提交：

```
git status
git add Source/HoyoGas/HoyoGas.Build.cs Source/HoyoGas/OrigamiBird
git commit -m "Add origami bird match core"
git push
```

12. 如果报错，优先检查

找不到头文件

检查 `HoyoGas.Build.cs` 是否加了：

```
Path.Combine(ModuleDirectory, "OrigamiBird/Public"),
Path.Combine(ModuleDirectory, "OrigamiBird/Private"),
```

generated.h 报错

确认 `.generated.h` 是最后一个 include。

HOYOGAS_API 未识别

确认文件在 `Source/HoyoGas` 模块内，不是在项目根目录。

动态委托 TArray 报错

如果 `DECLARE_DYNAMIC_MULTICAST_DELEGATE_TwoParams` 对 `const TArray<FIntPoint>&` 报错，把这一行：

```
DECLARE_DYNAMIC_MULTICAST_DELEGATE_TwoParams(FOrigamiBirdTilesMatchedEvent, const
TArray<FIntPoint>&, MatchedPositions, int32, ComboIndex);
```

临时改成：

```
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(FOrigamiBirdTilesMatchedEvent, int32,
MatchedTileCount);
```

然后 cpp 里：

```
OnTilesMatched.Broadcast(Matches.Num());
```

后面我们再设计 Blueprint 友好的事件 Payload。

13. 下一步

里程碑 1 编译通过后，我们下一步做：

1. GM / 快捷键测试入口
2. 更稳定的事件 **Payload**
3. 简单自动测试函数
4. **DataTable** 读关卡配置